

CENTRO UNIVERSITÁRIO SENAI SÃO PAULO

UNISENAI-SP CAMPUS SOROCABA - SANTA ROSÁLIA

ARTIGO CIENTÍFICO

**Toyota Experience: Full-Stack Platform Development
for Dealership Management with Microservices
Architecture**

Autores:

Bianca Isabelle Nunes

Eloisa Lopes Gomes

Julia Harumi Nascimento

Lauren Julia Reis Silva

Paola da Costa

Paola Oliveira

Sara Ferreira Rosmaninho Rosa

Vitor Henrique da Silva Fogaça

Sorocaba, SP - 2026

RESUMO

Este artigo apresenta o desenvolvimento do sistema Experience, uma plataforma digital voltada à gestão integrada de concessionárias automotivas e ao rastreamento do ciclo de fabricação de veículos. O trabalho descreve a arquitetura do backend desenvolvido em Java com Spring Boot, a implementação de controle de acesso baseado em papéis (RBAC) com autenticação JWT, e a modelagem de uma máquina de estados para rastreabilidade do processo produtivo. A solução integra os domínios de identidade, vendas e fabricação em uma API REST segura, containerizada com Docker e PostgreSQL. Os resultados demonstram que a plataforma elimina registros manuais do ciclo de fabricação, garante integridade referencial dos dados e estabelece uma base técnica para integração com dispositivos IoT e interfaces mobile. O trabalho evidencia a integração interdisciplinar entre as áreas de desenvolvimento backend, segurança da informação, planejamento estratégico de TI e integração de sistemas.

Palavras-chave: IoT Industrial, Spring Boot, JWT, Máquina de Estados, Rastreabilidade, Indústria 4.0, API REST.

1 INTRODUÇÃO

A transformação digital no setor automotivo impõe às concessionárias e fabricantes a necessidade de integrar processos historicamente isolados: cadastros de clientes, gestão de pedidos e acompanhamento da produção em uma cadeia de valor unificada e rastreável. No contexto da Indústria 4.0, a convergência entre Tecnologia da Informação (TI) e Tecnologia Operacional (TO) deixou de ser uma tendência para se tornar um requisito competitivo (**SINCLAIR, 2018**).

O problema central que motivou este trabalho foi a ausência de um sistema digital capaz de conectar o processo de venda ao ciclo de fabricação do veículo. Sem essa integração, clientes não têm visibilidade sobre o status do veículo adquirido, vendedores dependem de informações desatualizadas e gestores tomam decisões sem dados operacionais confiáveis. Adicionalmente, a ausência de controle de acesso adequado expõe dados sensíveis e compromete a conformidade com boas práticas de segurança da informação.

O objetivo deste trabalho é apresentar o desenvolvimento do sistema Experience, uma plataforma backend que resolve esses problemas por meio de uma API REST segura, com autenticação baseada em tokens JWT, controle de acesso por papéis (RBAC), máquina de estados para rastreabilidade do ciclo de fabricação e persistência em banco de dados relacional.

Este artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica; a Seção 3 descreve a metodologia adotada; a Seção 4 detalha o desenvolvimento da solução; a Seção 5 apresenta os resultados obtidos; a Seção 6 discute os resultados à luz da teoria; e a Seção 7 conclui o trabalho.

2 Fundamentação teórica

2.1 Planejamento Estratégico de TI e Alinhamento com o Negócio

O alinhamento entre TI e negócio é um dos pilares do Planejamento Estratégico de Tecnologia da Informação (PETI). Segundo Luftman (2003), o modelo de maturidade de alinhamento estratégico avalia em que medida a TI e o negócio operam como parceiros, compartilhando objetivos e métricas comuns. No nível mais elevado desse modelo, a TI deixa de ser suporte e passa a ser agente de geração de valor. O sistema Experience aplica esse princípio ao mapear os papéis organizacionais — cliente, vendedor, administrador e dispositivo IoT — diretamente em roles de acesso no sistema, fazendo com que a estrutura técnica reflita a hierarquia e as responsabilidades da organização.

2.2 Governança de TI

Weill e Ross (2004) definem governança de TI como o conjunto de decisões sobre quem tem autoridade para tomar decisões relacionadas ao uso dos recursos de tecnologia. Essa governança se materializa no sistema Experience por meio do controle de acesso baseado em papéis (RBAC), implementado com Spring Security e tokens JWT. Cada papel possui permissões específicas e não intercambiáveis: o papel IOT é o único autorizado a atualizar o status de fabricação de veículos, enquanto o papel ADMIN possui acesso irrestrito e o papel CLIENTE acessa exclusivamente seus próprios pedidos.

2.3 Engenharia de Software e Arquitetura

Sommerville (2011) estabelece que sistemas de software de qualidade devem ser construídos com baixo acoplamento e alta coesão, de modo que mudanças em uma parte do sistema não propaguem efeitos indesejados em outras. O sistema Experience aplica esse princípio por meio do padrão arquitetural DTO (Data Transfer Object) combinado com Mappers dedicados, garantindo que

as entidades JPA do domínio de negócio não sejam expostas diretamente pela camada de transporte da API.

2.4 Domain-Driven Design

Evans (2003) propõe que o software deve ser modelado em torno do domínio do negócio, protegendo as invariantes do modelo por meio de Aggregates e regras explícitas de transição de estado. O sistema Experience aplica esses conceitos na entidade Veiculo e no serviço StatusHistoricoService, que implementa uma máquina de estados com validação explícita de transições. O sistema rejeita ativamente qualquer tentativa de avançar o status de fabricação fora da sequência definida, garantindo que o estado do veículo no sistema sempre corresponda a uma progressão válida do processo produtivo.

2.5 Balanced Scorecard e Métricas Operacionais

Kaplan e Norton (1992) propõem o Balanced Scorecard como framework para conectar indicadores operacionais à estratégia corporativa. No sistema Experience, a entidade StatusHistorico registra cada transição de estado com timestamp preciso, criando uma trilha de auditoria completa do ciclo de vida de cada veículo. Esses dados permitem calcular o tempo médio em cada etapa do processo produtivo, transformando eventos operacionais em indicadores estratégicos mensuráveis.

2.6 Integração de Sistemas e IoT Industrial

A Internet Industrial das Coisas (IIoT) representa a aplicação dos princípios de IoT em ambientes produtivos de missão crítica, onde requisitos de confiabilidade, segurança e disponibilidade são significativamente mais rigorosos do que em aplicações de consumo (SINCLAIR, 2018). No contexto do sistema Experience, a integração com dispositivos IoT é viabilizada pelo papel IOT, que restringe o acesso

aos endpoints de atualização de status de fabricação a dispositivos previamente autenticados, estabelecendo uma interface segura entre o mundo físico e o sistema digital.

3. Metodologia

3.1 Tipo de Pesquisa

Este trabalho caracteriza-se como pesquisa aplicada de natureza experimental, com desenvolvimento de um sistema de software funcional como produto principal. A abordagem é qualitativa no que se refere à análise arquitetural e quantitativa na avaliação dos resultados técnicos obtidos.

3.2 Organização do Projeto

O desenvolvimento foi conduzido com base na metodologia ágil Scrum, com sprints organizadas em torno de entregas incrementais. A primeira sprint focou na definição da arquitetura e modelagem do domínio. A segunda sprint entregou o MVP funcional com API REST, autenticação JWT e CRUD dos domínios principais. As sprints subsequentes evoluíram o sistema com a implementação da máquina de estados, DTOs, mappers e containerização com Docker.

A equipe foi organizada de forma interdisciplinar, com responsabilidades distribuídas entre os domínios de backend, banco de dados e integração de sistemas.

3.3 Tecnologias Utilizadas

O sistema foi desenvolvido com as seguintes tecnologias:

- **Linguagem:** Java 17
- **Framework:** Spring Boot 3.3.1
- **Segurança:** Spring Security + JWT (jjwt 0.11.5) + BCryptPasswordEncoder
- **Persistência:** Spring Data JPA + PostgreSQL 16 (produção) + H2 (testes)
- **Containerização:** Docker Compose
- **Redução de boilerplate:** Lombok 1.18.36
- **Validação:** Bean Validation com suporte a CPF e CNPJ brasileiros (Hibernate Validator)
- **Autenticação OAuth2:** Spring Boot OAuth2 Client (disponível para extensão futura)

3.4 Arquitetura Geral

A arquitetura do sistema segue o padrão em camadas, organizado nos seguintes pacotes:

...

controllers → recebem requisições HTTP e delegam para services

services → contêm a lógica de negócio

repositories → abstraem o acesso ao banco de dados (Spring Data JPA)

entities → modelo de domínio persistido

DTO → objetos de transferência de dados (request/response)

mappers → convertem entre entidades e DTOs

security → filtros JWT, configuração de acesso e autenticação

...

Essa separação garante que cada camada tenha uma responsabilidade única e bem definida, facilitando manutenção e evolução do sistema.

4. Desenvolvimento da Solução

4.1 Domínio de Identidade

O domínio de identidade é a base do sistema. A entidade `Usuario` utiliza herança com estratégia `JOINED` no JPA, permitindo que `PessoaFisica` e `PessoaJuridica` herdem os atributos comuns — nome, email, `senhaHash`, `dataNascimento` e `role` — enquanto mantêm seus atributos específicos em tabelas separadas.

A entidade `PessoaFisica` armazena o CPF com validação brasileira via anotação `@CPF` do `Hibernate Validator`, garantindo que apenas documentos válidos sejam aceitos. Da mesma forma, `PessoaJuridica` valida o CNPJ e a Razão Social. Essa validação ocorre na camada de domínio, antes de qualquer persistência no banco de dados.

O campo `role` na entidade `Usuario` armazena o papel do usuário como enum `UserRole`, com os valores `CLIENTE`, `VENDEDOR`, `ADMIN` e `IOT`. Esse campo é persistido como string no banco de dados e utilizado pelo sistema de autenticação para determinar as permissões de cada usuário.

4.2 Autenticação e Controle de Acesso

A autenticação é implementada com tokens JWT stateless. O endpoint POST /api/usuario/login recebe email e senha, valida as credenciais com BCryptPasswordEncoder e, em caso de sucesso, retorna um token JWT assinado com HMAC-SHA256 contendo o email do usuário e seu papel como claims. O token tem validade de 24 horas.

O filtro JwtAuthFilter intercepta todas as requisições, extrai o token do cabeçalho Authorization, valida a assinatura e popula o contexto de segurança do Spring com as autoridades correspondentes ao papel do usuário. O segredo JWT é injetado via variável de ambiente (@Value("\${jwt.secret}")), nunca armazenado no código-fonte.

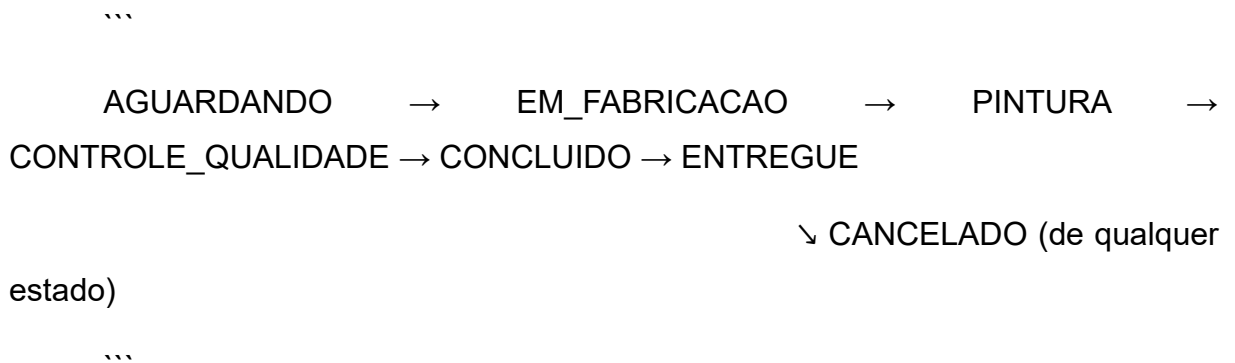
As políticas de acesso são definidas no SecurityConfig com as seguintes regras:

- POST /api/fabricacao/** e POST /api/veiculo/*/status → exclusivo para role IOT
- /api/admin/** → exclusivo para role ADMIN
- /api/pedido/meus-pedidos → roles CLIENTE, VENDEDOR e ADMIN
- /api/pedido/** → roles VENDEDOR e ADMIN
- Operações de escrita em /api/produto/** → roles ADMIN e VENDEDOR

Essa configuração garante que cada ator do sistema acesse apenas os recursos compatíveis com sua função organizacional.

4.3 Domínio de Fabricação e Máquina de Estados

O domínio de fabricação é o núcleo técnico mais relevante do sistema. A entidade Veiculo possui um campo statusVeiculo do tipo enum StatusFabricacao, que define os seguintes estados possíveis:



A transição entre estados é controlada pelo StatusHistoricoService, que implementa uma máquina de estados com validação explícita via estrutura switch. As transições válidas são:

- AGUARDANDO → EM_FABRICACAO ou CANCELADO
- EM_FABRICACAO → PINTURA ou CANCELADO
- PINTURA → CONTROLE_QUALIDADE ou CANCELADO
- CONTROLE_QUALIDADE → CONCLUIDO ou CANCELADO
- CONCLUIDO → ENTREGUE
- ENTREGUE e CANCELADO → nenhuma transição permitida

Qualquer tentativa de transição inválida resulta em RuntimeException com mensagem descritiva. Isso garante que o estado do veículo no sistema nunca represente uma progressão fisicamente impossível do processo produtivo.

Cada transição válida gera um registro na entidade `StatusHistorico`, que armazena o veículo de referência, o novo status e o timestamp exato da alteração (`dataAlteracao` como `LocalDateTime`). Essa trilha de auditoria é consultável via `GET /api/veiculo/{id}/status`, retornando o histórico completo ordenado por data decrescente.

4.4 Domínio de Vendas

O domínio de vendas conecta os demais domínios do sistema. A entidade `Pedido` referencia dois objetos `Usuario` — um como cliente (`idCliente`) e outro como vendedor (`idVendedor`) — por meio de chaves estrangeiras reais (`@ManyToOne` com `@JoinColumn`), garantindo integridade referencial no banco de dados.

O `PedidoService` implementa o método `findMeusPedidos`, que filtra os pedidos com base no papel do usuário autenticado: `ADMIN` visualiza todos os pedidos, `VENDEDOR` visualiza os pedidos em que é o responsável pela venda, e `CLIENTE` visualiza apenas os pedidos em que é o comprador. Essa lógica integra o domínio de identidade ao domínio de vendas de forma coesa.

A entidade `ItemPedido` associa um `Pedido` a um `Produto` com quantidade, permitindo que um pedido contenha múltiplos itens. A entidade `Produto` armazena modelo, cor, versão e ano do veículo.

4.5 Padrão DTO e Mappers

Todos os endpoints da API utilizam DTOs de `Request` e `Response` em vez de expor diretamente as entidades JPA. Essa decisão arquitetural tem dois benefícios

principais: protege o modelo de domínio de alterações externas e permite que a API evolua independentemente da estrutura do banco de dados.

Os Mappers são classes utilitárias estáticas que convertem entre entidades e DTOs. Por exemplo, o `UsuarioMapper` converte um `UsuarioRequest` em uma entidade `Usuario` (incluindo o mapeamento do campo `role`) e converte uma entidade `Usuario` em um `UsuarioResponse` (excluindo o campo `senhaHash` da resposta).

4.6 Infraestrutura e Containerização

O ambiente de produção é orquestrado com Docker Compose, que provisiona dois serviços: PostgreSQL 16 com banco de dados `db_experience` e pgAdmin 4 para administração visual do banco. Os dados do PostgreSQL são persistidos em volume Docker, garantindo que não sejam perdidos entre reinicializações do container.

O CORS está configurado para aceitar requisições de dois frontends simultâneos (`localhost:3000` e `localhost:5173`), indicando compatibilidade com aplicações React e Vite. A dependência Spring Boot OAuth2 Client está disponível no `pom.xml` para extensão futura com provedores de identidade externos.

5. Resultados

5.1 API REST Funcional

O sistema disponibiliza os seguintes endpoints funcionais, todos protegidos por autenticação JWT:

| Domínio | Endpoints |

|---|---|

| Usuário | GET, POST, PUT, DELETE /api/usuario; POST /api/usuario/login;
GET /api/usuario/me |

| Pessoa Física | GET, POST, PUT, DELETE /api/pessoaFisica |

| Pessoa Jurídica | GET, POST, PUT, DELETE /api/pessoaJuridica |

| Endereço | GET, POST, PUT, DELETE /api/endereco |

| Telefone | GET, POST, PUT, DELETE /api/telefones |

| Produto | GET, POST, PUT, DELETE /api/produto |

| Veículo | GET, POST, PUT, DELETE /api/veiculo |

| Status de Fabricação | GET, POST /api/veiculo/{id}/status |

| Pedido | GET, POST, PUT, DELETE /api/pedido; GET /api/pedido/meus-
pedidos |

| Item de Pedido | GET, POST, PUT, DELETE /api/itemPedido |

5.2 Controle de Acesso por Papel

O sistema implementa quatro papéis distintos com políticas de acesso verificadas em cada requisição. A autenticação via POST /api/usuario/login retorna um objeto LoginResponse contendo o token JWT, o email do usuário e seu papel, permitindo que o cliente da API adapte sua interface com base nas permissões disponíveis.

5.3 Rastreabilidade do Ciclo de Fabricação

A máquina de estados implementada no StatusHistoricoService garante que cada veículo percorra o ciclo de fabricação de forma controlada e auditável. O endpoint GET /api/veiculo/{id}/status retorna a lista completa de transições com

timestamps, permitindo calcular o tempo decorrido em cada etapa do processo produtivo.

5.4 Integridade dos Dados

A combinação de validações de CPF e CNPJ na camada de domínio, chaves estrangeiras reais entre entidades, enums para estados e métodos de pagamento, e BCrypt para senhas garante que dados inconsistentes não sejam persistidos no banco de dados. A máquina de estados adiciona uma camada adicional de integridade ao nível do processo de negócio.

5.5 Ambiente Reproduzível

O Docker Compose permite que qualquer membro da equipe ou avaliador reproduza o ambiente completo com um único comando, eliminando dependências de configuração manual e garantindo consistência entre ambientes de desenvolvimento e produção.

6. Discussão

6.1 Alinhamento com os Objetivos Estratégicos

Os resultados obtidos demonstram alinhamento com os princípios de governança de TI propostos por Weill e Ross (2004). A implementação de RBAC com quatro papéis distintos reflete diretamente a estrutura organizacional de uma concessionária automotiva, onde clientes, vendedores, administradores e dispositivos industriais possuem responsabilidades e acessos claramente diferenciados. Essa

correspondência entre estrutura organizacional e estrutura técnica é o que Luftman (2003) denomina alinhamento estratégico de alto nível de maturidade.

6.2 Contribuição da Máquina de Estados

A implementação da máquina de estados no StatusHistoricoService vai além de uma funcionalidade técnica — ela representa a digitalização de um processo físico real. Ao rejeitar transições inválidas e registrar cada mudança com timestamp, o sistema cria um ativo de dados que pode ser utilizado para análise de desempenho operacional, identificação de gargalos e tomada de decisão baseada em evidências, conforme preconizado por Kaplan e Norton (1992) no contexto do Balanced Scorecard.

6.3 Preparação para Integração IoT

A existência do papel IOT no sistema, com restrições de acesso exclusivas nos endpoints de fabricação, demonstra que a arquitetura foi projetada para receber eventos de dispositivos físicos. Um dispositivo IoT — como um gateway industrial ou microcontrolador com conectividade HTTP — poderia autenticar-se via POST /api/usuario/login com credenciais de um usuário com papel IOT e, a partir daí, atualizar o status de fabricação dos veículos de forma automatizada. Essa integração não requer alterações na arquitetura existente, apenas a configuração do dispositivo físico.

6.4 Limitações

O sistema, em sua versão atual, apresenta as seguintes limitações que representam oportunidades para trabalhos futuros:

- **Ausência de frontend:** a API está pronta para consumo, mas não existe interface web ou mobile implementada. O CORS configurado para dois frontends indica que essa integração está prevista.

- **Integração com sistemas externos:** não há integração com sistemas ERP ou CRM. A arquitetura baseada em DTOs e serviços independentes facilita essa extensão futura.

- **Protocolo IoT dedicado:** a integração com dispositivos IoT ocorre via HTTP REST. A adoção de protocolos específicos como MQTT poderia reduzir a latência e o consumo de recursos em dispositivos com capacidade computacional limitada.

- **Ausência de testes automatizados:** o projeto contém apenas o teste de contexto padrão do Spring Boot. A implementação de testes unitários e de integração aumentaria a confiabilidade do sistema.

6.5 Integração Interdisciplinar

O projeto demonstra integração efetiva entre múltiplas disciplinas. O backend em Spring Boot conecta-se diretamente com os conceitos de integração de sistemas e IoT Industrial. A implementação de JWT e RBAC aplica princípios de segurança da informação e governança de TI. A máquina de estados e os DTOs refletem conceitos de engenharia de software e DDD. A containerização com Docker e a configuração de CORS para múltiplos frontends evidenciam a preparação para integração com camadas de cloud e mobile.

Referências

EVANS, Eric. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. Boston: Addison-Wesley, 2003.

KAPLAN, Robert S.; NORTON, David P. The Balanced Scorecard: Measures That Drive Performance. **Harvard Business Review**^{**}, v. 70, n. 1, p. 71-79, 1992.

LUFTMAN, Jerry. Assessing IT/Business Alignment. **Information Systems Management**, v. 20, n. 4, p. 9-15, 2003.

SINCLAIR, Bruce. **IoT: como usar a internet das coisas para alavancar seus negócios**. São Paulo: Autêntica Business, 2018.

SOMMERVILLE, Ian. **Engenharia de Software**. 9. ed. São Paulo: Pearson, 2011.

WEILL, Peter; ROSS, Jeanne W. **IT Governance: How Top Performers Manage IT Decision Rights for Superior Results.** Boston: Harvard Business School Press, 2004.